ECE 220 Computer Systems & Programming

Lecture 17 — Linked Lists

October 22, 2026

Prof. Sayan Mitra mitras@illinois.edu
Course website: courses.grainger.illinois.edu/ece220/fa2026

ILLINOIS
Electrical & Computer Engineering
GRAINGER COLLEGE OF ENGINEERING

A Late Enrollment

```
int insert_sorted(student ece220[], int n,
                  student s) {
    int i = n;
    while (i > 0 && ece220[i - 1].UIN > s.UIN) {
        ece220[i] = ece220[i - 1];  /* move up */
        i--;
    }
    ece220[i] = s;
    return n - i;
}
int main() {
    student ece220[200] = {
        {"Jamie", 123456789, 3.89},
        {"Robin", 246813579, 3.52},
        {"Alex",  987654321, 3.98}};
    student casey = {"Casey", 135792468, 3.71};
    int n = 3, moved, i;

    moved = insert_sorted(ece220, n, casey);
    n++;
```

```
Jamie 123456789 3.89
Casey 135792468 3.71
Robin 246813579 3.52
Alex 987654321 3.98
2 records moved, 108 bytes each
```

200 students, and a UIN below all of them: **199** records, **21492** bytes copied, for one new student.

A linked list adds Casey by changing two pointers. No record moves.

A Late Enrollment

```
int insert_sorted(student ece220[], int n,
                  student s) {
    int i = n;
    while (i > 0 && ece220[i - 1].UIN > s.UIN) {
        ece220[i] = ece220[i - 1];  /* move up */
        i--;
    }
    ece220[i] = s;
    return n - i;
}
int main() {
    student ece220[200] = {
        {"Jamie", 123456789, 3.89},
        {"Robin", 246813579, 3.52},
        {"Alex",  987654321, 3.98}};
    student casey = {"Casey", 135792468, 3.71};
    int n = 3, moved, i;

    moved = insert_sorted(ece220, n, casey);
    n++;
```

ece220 is sorted by UIN. Casey, UIN 135792468, enrolls after the roster was built. The program prints

```
Jamie 123456789 3.89
Casey 135792468 3.71
Robin 246813579 3.52
Alex 987654321 3.98
2 records moved, 108 bytes each
```

200 students, and a UIN below all of them: **199** records, **21492** bytes copied, for one new student.

A linked list adds Casey by changing two pointers. No record moves.

Linked List Data Structure

1. A **data structure** defines how data is organized and accessed.

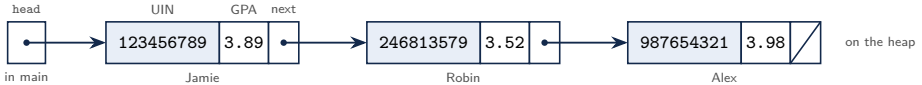
2. A data structure comes hand in hand with the **algorithms** that create it, access it, operate on it, and destroy it.

3. **What is a linked list, and what are the algorithms that create one, access it, operate on it, and destroy it?**

`find_node`, `add_node` and `remove_node` are each recursive: a base case, then the same call on the rest of the list.

The Linked List Data Structure

A **linked list** is an ordered collection of **nodes**, each holding some data, joined by pointers: each node holds the address of the next node.




```
typedef struct studentStruct Node;
struct studentStruct {
    int   UIN;
    float GPA;
    Node *next;
};
```

The `typedef` comes first because the struct uses the name `Node` inside its own definition.

- The first node is the **head**; the variable `head` holds **the first node's address**
- The last node is the **tail**; its `next` is **NULL**, drawn as a slash
- The empty list: `head` == **NULL**
- `sizeof(Node)` on the laptop: **16**: $4 + 4 + 8$
- The nodes live **on the heap**; `head` lives in `main`'s activation record (AR) on the run-time stack (RTS)

Array vs. Linked List

	Array `student ece220[200]`	**Linked list** `Node *head`
Memory allocation	one block for all 200, by a declaration or by one `malloc`	one `malloc` per node, as each record is added
Memory structure	contiguous: element `i` at the start + $i \times 108$	nodes anywhere on the heap; `next` holds the order
Memory overhead	none: 108 bytes per student	one pointer per node: 8 of the 16 bytes
Order of access	random: `ece220[i]` in one step, for any `i`	sequential: from `head`, i steps to node i
Insertion / deletion	shift every record after the spot	change two pointers, or one; no record moves

Binary search needs random access. A sorted linked list is searched from the head, one node at a time.

Build a List by Hand

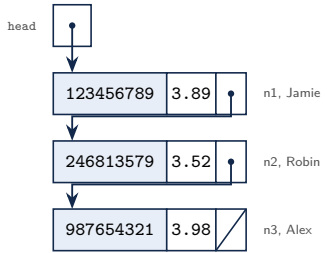
```
Node *head, *n1, *n2, *n3;

n1 = (Node *)malloc(sizeof(Node)); /* Jamie */
n2 = (Node *)malloc(sizeof(Node)); /* Robin */
n3 = (Node *)malloc(sizeof(Node)); /* Alex  */
if (n1 == NULL || n2 == NULL || n3 == NULL)
    return 1;
n1->UIN = 123456789;  n1->GPA = 3.89;
n2->UIN = 246813579;  n2->GPA = 3.52;
n3->UIN = 987654321;  n3->GPA = 3.98;
n1->next = n2;
n2->next = n3;
n3->next = NULL;
head = n1;

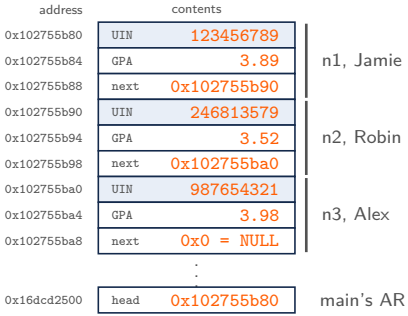
print_list(head);
printf("second GPA: %.2f\n", head->next->GPA);
```

- `head->UIN` is **123456789**
- `head->next->next->GPA` is **3.98**
- `head->next->next->next` is **NULL**; the last line prints **3.52**

Draw `head` and the three nodes after `head = n1;`



The List in Memory



Lower addresses at the top. One run of `build.c` on a Mac; the addresses change from run to run.

`build.c` prints the addresses:

```
sizeof(Node) = 16
n1 at 0x102755b80  next 0x102755b90
n2 at 0x102755b90  next 0x102755ba0
n3 at 0x102755ba0  next 0x0
head at 0x16dcd2500 holds 0x102755b80
```

- One node is 16 bytes: `UIN` at +0, `GPA` at +4, `next` at +8
- `n1->next` holds **n2's address**: a `next` field is an address, as `head` is
- `n2->GPA` is at **0x102755b94**
- `malloc` placed these nodes 16 bytes apart. After a `free` and a new `malloc`, a node can land anywhere; `next` alone keeps the order.

Walk the List: `print_list`, Two Ways

With a loop (`build.c`)

```
void print_list(Node *head) {
    Node *current = head;
    while (current != NULL) {
        printf("%d %.2f\n",
               current->UIN, current->GPA);
        current = current->next;
    }
}
```

- `current` starts at **head**; the loop runs while **current != NULL**
- One step: **current = current->next**
- `head` itself **never changes**: `main` still has the list

Walk the List: `print_list`, Two Ways

With a loop (`build.c`)

```
void print_list(Node *head) {
    Node *current = head;
    while (current != NULL) {
        printf("%d %.2f\n",
               current->UIN, current->GPA);
        current = current->next;
    }
}
```

- `current` starts at **head**; the loop runs while **current != NULL**
- One step: **current = current->next**
- `head` itself **never changes**: `main` still has the list

Recursively (`llist.c`)

```
void print_list(Node *head) {
    if (head == NULL)
        return;
    printf("%d %.2f\n",
           head->UIN, head->GPA);
    print_list(head->next);
}
```

- Base case: **head == NULL**, the empty list
- Recursive case: print this node, then the same call on **head->next**
- Calls for the three-node list: 4

A list is one node followed by a shorter list, or NULL. Every function today has that shape.

Both print `123456789 3.89`, `246813579 3.52`, `987654321 3.98`, one per line.

find_node: Search the Sorted List

```
Node *find_node(Node *head, int S_UIN) {
    /* base case 1: list end, or past S_UIN */
    if (head == NULL || head->UIN > S_UIN) {
        printf("record not found\n");
        return NULL;
    }
    /* base case 2: this node matches S_UIN */
    if (head->UIN == S_UIN) {
        printf("record found\n");
        return head;
    }
    /* recursive case: the remaining list */
    return find_node(head->next, S_UIN);
}
```

It returns the node's address, so `main` can read or change the record: `p = find_node(head, 246813579);` then `p->GPA`.

Trace `find_node(head, 246813579)`

- call 1: `head` is Jamie's node; 123456789 $<$ 246813579: **recursive case**
- call 2: `head` is Robin's node: **base case 2, found**; it returns **Robin's address**
- call 1 returns **what call 2 returned**

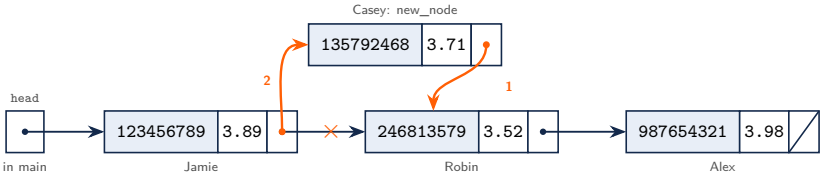
Trace `find_node(head, 200000000)`

- call 2: 246813579 $>$ 200000000: **base case 1, not found**; Alex's node is **never visited**
- A UIN above every UIN in a list of n nodes: **$n + 1$** calls

Base case 1 uses the sorting: after a UIN above `S_UIN`, no node can match.

Add Casey: Which Pointers Change?

`add_node(&head, 135792468, 3.71)`: Casey's UIN falls between Jamie's and Robin's. Draw the pointers after the call.



- 1: `new_node->next =` **Jamie's next**, Robin's address
- 2: **Jamie's next = new_node**
- Step 1 comes first: otherwise **Robin's address is lost**

- A UIN below 123456789 goes before Jamie. The pointer that changes is **head, in main's AR**
- So `main` passes `&head`. `Node **list` holds the address of a pointer: `main`'s `head` on the first call, **a node's next field** on each recursive call. `*list = new_node` writes into whichever it is.

In-Class Problem: `add_node`

Base case 1 covers the empty list, a UIN below the first node's, and every spot in the middle: the new node goes in front of `*list`.

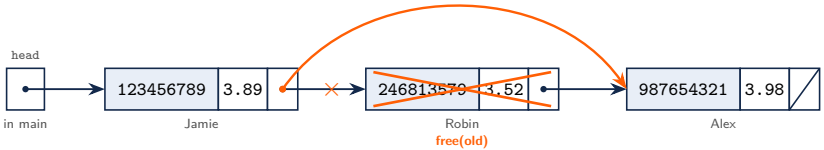
- Its test: `*list == NULL ||` **`(*list)->UIN > new_UIN`**
- The UIN exists: **return, add nothing**
- The recursive call passes **`&(*list)->next`**: this node's `next` field
- `main` adds Alex, Jamie, Robin in that order; then `head` points at **Jamie**

`new_node->next = *list` comes before `*list = new_node`, or the node `*list` held is lost.

```
void add_node(Node **list,
              int new_UIN, float new_GPA) {
    Node *new_node;
    /* base case 1: insert new node at 'head' */
    if (*list == NULL || (*list)->UIN > new_UIN) {
        new_node = (Node *)malloc(sizeof(Node));
        if (new_node == NULL) return;
        new_node->UIN = new_UIN;
        new_node->GPA = new_GPA;
        new_node->next = *list;
        *list = new_node;
        return;
    }
    /* base case 2: UIN already exists */
    if ((*list)->UIN == new_UIN) return;
    /* recursive case: the remaining list */
    add_node(&(*list)->next, new_UIN, new_GPA);
}
```

Remove Robin: Which Pointer Changes?

`remove_node(&head, 246813579)`. Draw the pointers after the call, and mark the node that is freed.



- The pointer that changes: **Jamie's next**; its new value: **Alex's address**
- Then **free Robin's node**. Its address is saved in `old` first, because afterwards ***list no longer holds it**

- Remove Jamie instead: the pointer that changes is **head**. The same two lines do it, because `*list` is `head` on the first call
- `remove_node(&head, 200000000)`: the walk stops at **Robin**, base case 1: not found; nothing is freed

In-Class Problem: `remove_node`

- Base case 1 is `find_node`'s test, written with `*list`: `*list == NULL || (*list)->UIN > old_UIN`
- Base case 2, in order: save `*list` in `old`; set `*list` to `old->next`; then `free(old)`
- `main` removes Robin twice. The second call prints **record not found**
- `free_list` frees **the rest first**, then this node: after `free(head)`, `head->next` must not be read

`leaks -atExit - ./llist` reports 0 leaks.

```
void remove_node(Node **list, int old_UIN) {
    Node *old;
    /* base case 1: list end, or past old_UIN */
    if (*list == NULL || (*list)->UIN > old_UIN) {
        printf("record not found\n");
        return;
    }
    /* base case 2: found: unlink and free it */
    if ((*list)->UIN == old_UIN) {
        old = *list;
        *list = old->next;
        free(old);
        return;
    }
    /* recursive case: the remaining list */
    remove_node(&(*list)->next, old_UIN);
}
```

Summary & What's Next

Today

- **The problem**: a late enrollment in a sorted array shifts every record after it
- **Nodes and links**: `Node` holds `UIN`, `GPA` and `next`; `head` holds the first node's address; the tail's `next` is `NULL`; one `malloc` per node, nodes anywhere on the heap, 8 of 16 bytes for the pointer
- **Build and walk**: three `malloc`s and three `next` assignments; `print_list` with `current = current->next`, or recursively on `head->next`
- **find_node**: base case 1 (list end, or past `S_UIN`: NULL), base case 2 (match: this node), else the same call on `head->next`
- **add_node** and **remove_node**: `Node **list` holds the address of `head` or of a node's `next`; inserting sets two pointers, removing sets one and frees; no record moves; `leaks` reports 0

Next lecture

- Problem solving with linked lists: `add_node(&head, ...)` drawn on the RTS; doubly linked lists; a stack and a queue on a linked list

Code from today: `insert_array.c`, `build.c`, `llist.c` (skeleton `llist_skel.c`).